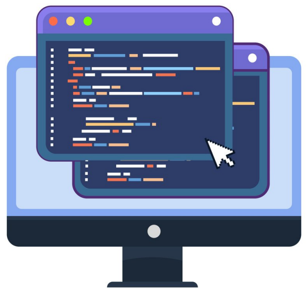
Welcome

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Sani Yusuf
Lead Software Engineering Content Developer

My experience with Java

- Working with Java since 2010
  - Algorithms
  - Android mobile applications
- Course created in collaboration with **Miller Archury**
  - Senior Software Engineer at Google
  - Former Microsoft Engineer



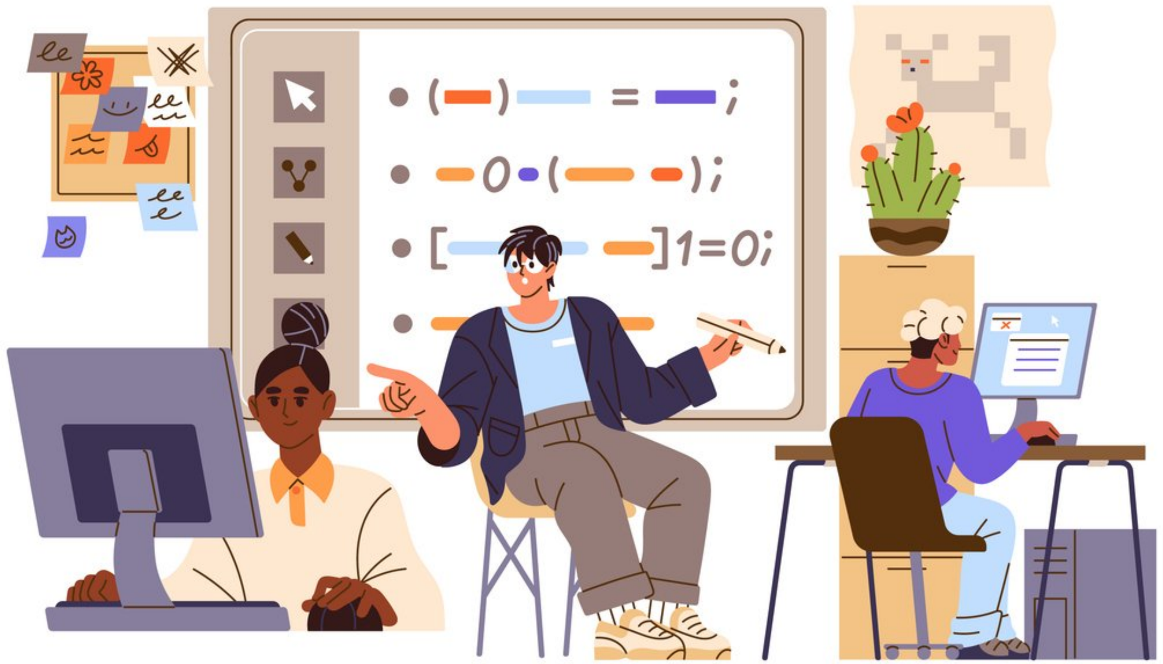
Microsoft

Google

[1] https://blog.google/press/

Topics covered

- Explore Java's approach to Object-Oriented Programming
- Simplify the complexities of Object-Oriented Programming

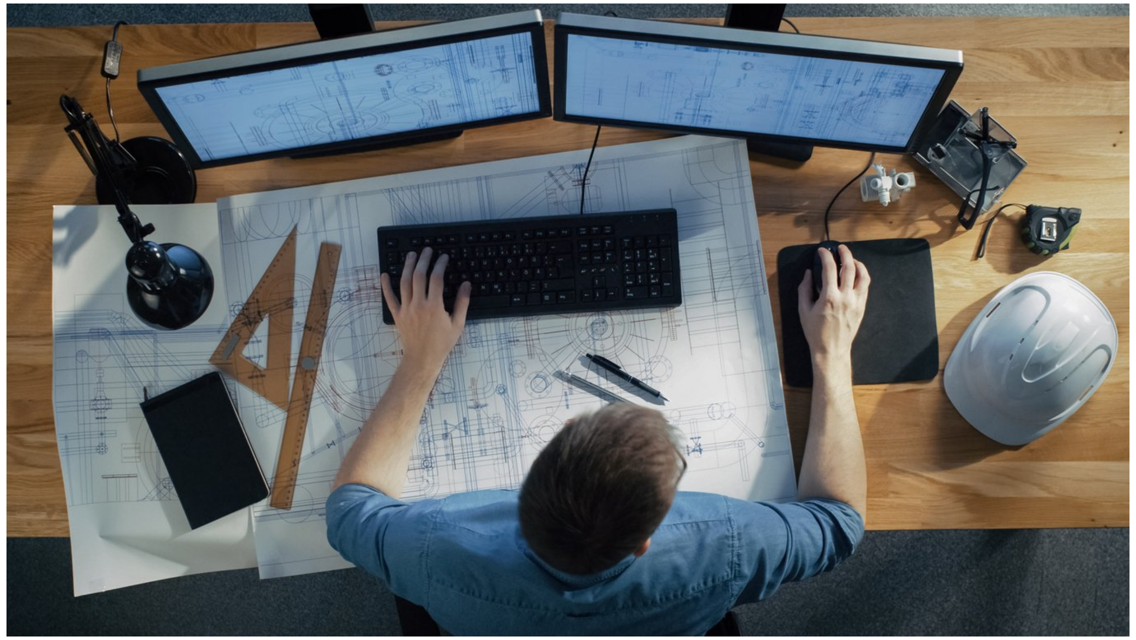


What is object oriented programming (OOP)

> OOP is a programming paradigm that models code based on objects and real-world items

What is a class?

- Classes are like Cookie cutters
- Cookies created take the shape of the cookie cutter
- Classes are blueprints for our code



Structure of a class

```java
// Cookie Class
class Cookie {
  // Cookie class's code goes here
}
```

Naming object instance

- **NOTE:** The `Main` **class** and `main` **method** will be used throughout this course
- All code will be inside the `Main` class
  - The `main` method is the entry point of all Java code

```
public static class Main {

  class Cookie {

  }

  public static void main(String[] args) {
    // Give the object instance a name
     Cookie myCookie = new Cookie();
    }
}
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Adding properties

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

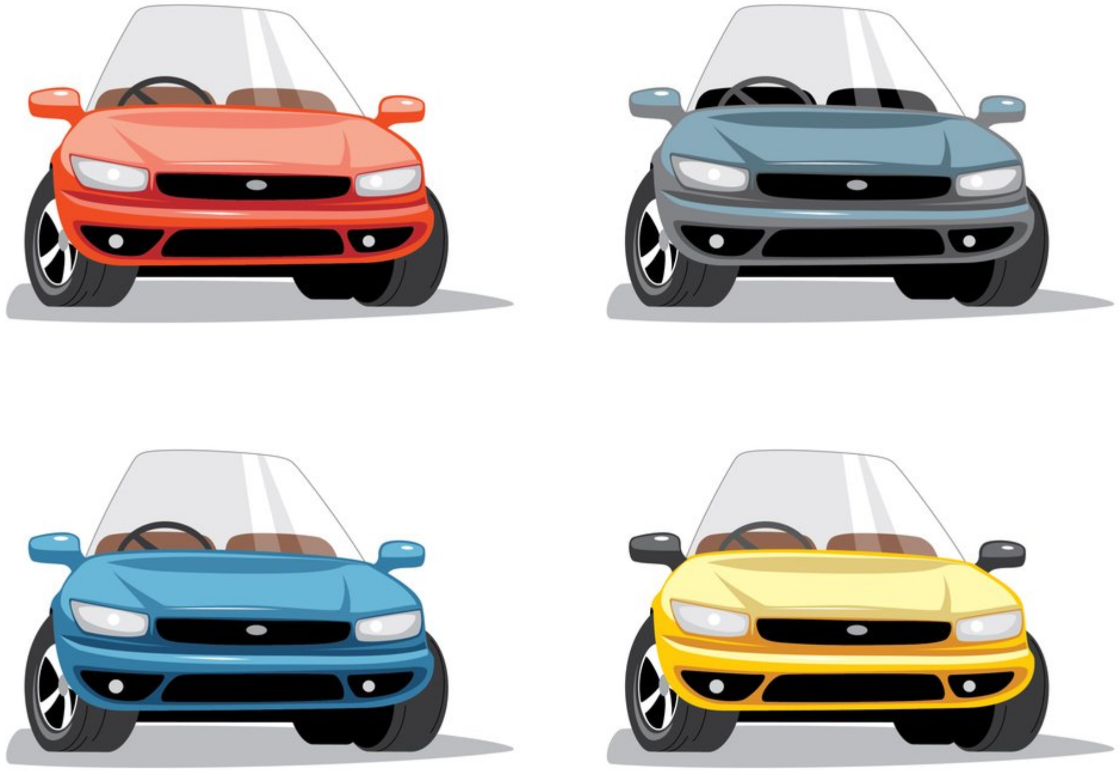
Sani Yusuf
Lead Software Engineering Content Developer

Car example

- Cars on the road can come in different forms, with features like color and model that differentiate them.

Differentiate cars

- Even when cars are the same, they differ by **color, license plate, vehicle registration.**



Adding properties to class

- Properties describe a class
- Properties can have different data types

```java
// Car class
class Car {
  String model; // Property for model of car
  int topSpeed; // Property for car's top speed
  boolean isInsured; // Property for current insurance state
}
```

Constructors in Java

- The Constructor is a method, always called when an object instance of a class is created
- Constructor must have the same name as the class it is in

```
class Passport {
  String firstName; // Passport holder's first name
  String lastName; // Passport holder's last name

  Passport() {
    // Constructor of Passport class
  }
}
```

Setting properties inside the constructors

```java
class Passport {
  String firstName; // Passport holder's first name
  String lastName; // Passport holder's last name

  Passport() {
    this.firstName = "David";  // Set property inside constructor
    this.lastName = "Beckham";
  }
}
```

Constructors with parameters

- Constructors can have parameters just like any other method

```java
class Passport {
  String firstName;
  String lastName;

  // Constructor with parameters
  Passport(String firstName, String lastName) {

  }
}
```

Setting properties with constructor parameters

- The `this` keyword refers to the `Passport` object
- It is common to give constructor parameters same name as class properties

```java
class Passport {
  String firstName;
  String lastName;

  Passport(String firstName, String lastName) {
        this.firstName = firstName; // Setting class properties
        this.lastName = lastName;   //  with constructor parameters
  }
}
```

Creating object instances with parameters

- Objects are created with the `new` keyword
- Constructor parameters are passed during object creation

```java
// Passport Class with constructor
class Passport {
  String firstName;
  String lastName;

  // Constructor method
  Passport(String firstName,
           String lastName){
        this.firstName = firstName;
        this.lastName = lastName;
  }
}
```

```java
// Main Class
public class Main {
  // main method (program entry point)
  public static void main(
    String[] args) {

    // Passing parameters to constructor
    // Passport(firstName, lastName)
    Passport myPassport =
      new Passport("Michael","Jackson");
    System.out.println(
      myPassport.firstName); // Michael
  }
}
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Class methods

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Sani Yusuf
Lead Software Engineering Content Developer

Example of car properties

- Cars can have colors.
- Each car has a model.
- Cars can also perform actions like go on/off.
- Cars can also give us data like a list of devices paired with the car's Bluetooth.

Types of methods

- **Void methods:** Methods that do not return data
- **Non-Void methods:** Returns data back

Void methods

- Void methods do not return any data
- They are marked with the keyword `void`

```java
// Car class
class Car {

    void honkHorn() {
     // Code for horn to sound
      System.out.println("beep beep");
     }
}
```

```java
// Main class
public class Main {
  public static void main(
    String[] args) {

    Car myNewCar = new Car();
    myNewCar.honkHorn(); // beep beep
  }
}
```

Creating non void methods

- Non void methods always have a return type
- Non void methods use the `return` keyword to return data

```
// Formula class
class Formula {

    // Non void method
    int getSquare(int number) {
        // Return an "int" value
      return number * number;
    }

}
```

```
// Main class
public class Main {
  // main method
  public static void main(
    String[] args) {

    Formula myFormula =
      new Formula();
    // Calling "getSquare" method
    System.out.println(
      myFormula.getSquare(5)); // 25
  }
}
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA